KPR Institute of Engineering and Technology

EXPLORATORY DATA ANALYSIS AND DEEP LEARNING FOR HUMAN ACTIVITY RECOGNITION

Project Report

Name: **PRAGHATHISH S**
Roll Number: **23AM045**
Department: CSE (AI&ML)
Academic Year: 2026 - 2027

Abstract

This project investigates human activity recognition using the UCI Human Activity Recognition Using Smartphones dataset. The dataset contains engineered features derived from smartphone accelerometer and gyroscope measurements and six activity labels. Exploratory data analysis is used to inspect data quality, class frequencies, feature distributions, potential outliers, correlations, and relationships among sensor features. Preprocessing includes a stratified validation split, training-only median imputation where required, and standardization fitted on training data. A Multilayer Perceptron (MLP) classifier is trained with basic validation-based hyperparameter selection and early stopping. The final model is evaluated on the official test partition using accuracy, class-wise precision/recall/F1, a confusion matrix, and one-vs-rest ROC-AUC. [After running the notebook, insert measured results and two or three key data-specific findings here.]

1. Introduction and Objectives

Human activity recognition aims to infer a person's physical activity from sensor measurements. Smartphone sensors provide a practical source of movement data for fitness applications, context-aware mobile systems, and activity-aware interfaces. The objective of this study is to explore the structure and statistical properties of the dataset, visualize important patterns, train a deep learning classifier, and evaluate its performance without using the test set for model selection.

Specific objectives:

- Understand the dataset fields, dimensions, and target classes.
- Inspect missing values, duplicate records, outliers, and feature distributions.
- Create at least five meaningful EDA visualizations and interpret them.
- Analyze discrete class frequencies and continuous feature distributions.
- Train an MLP model and perform basic hyperparameter comparison.
- Evaluate performance using training curves, confusion matrix, ROC curves, and class-wise metrics.
- Document limitations, future improvements, and reproducible code.

2. Dataset Description

Dataset name: Human Activity Recognition Using Smartphones. Source: UCI Machine Learning Repository. Dataset URL: https://archive.ics.uci.edu/dataset/240/humanactivityrecognitionusingsmartphones

The dataset contains 561 engineered features computed from accelerometer and gyroscope signals and six target activities: walking, walking upstairs, walking downstairs, sitting, standing, and laying. The notebook prints the exact loaded dimensions and class counts at runtime. Insert the values from your execution below.

- Training partition rows: (7352, 561)
- Official test partition rows: (2947, 561)
- Number of input features: 561
- Missing feature cells in training partition: 0
- Exact duplicate feature rows in training partition: 0

3. Exploratory Data Analysis and Preprocessing

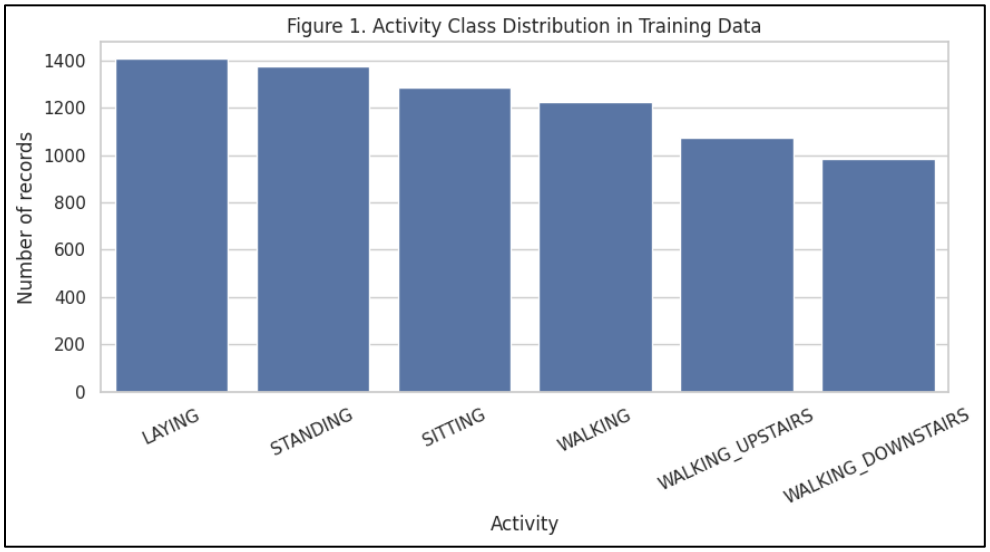
The analysis begins with dimensions, data types, descriptive statistics, missing-value counts, and exact duplicate-row counts. Histograms and skewness describe continuous feature distributions. The activity target is discrete and categorical, so it is summarized with class counts and proportions. Boxplots flag possible outliers using the 1.5×IQR rule; such points

are not automatically removed because extreme sensor-derived values may be valid. A correlation heatmap explores linear relationships among selected features, while a scatter plot explores whether two features show activity-specific separation.

The official training partition is split into training and validation subsets using stratification. The official test partition is preserved for final evaluation. If missing values exist, medians are learned from the training split only. StandardScaler is also fitted on training data only to avoid data leakage.

4. Data Visualization

Figure 1. Activity class distribution

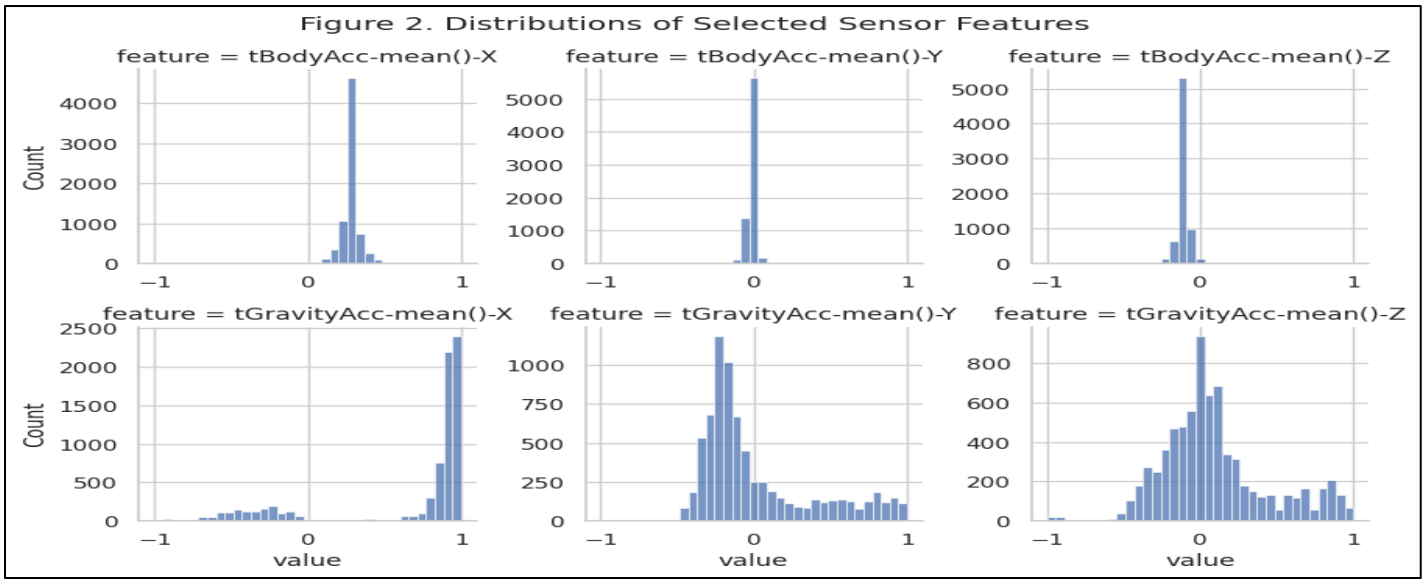


What it shows: number of training records in each activity class.

Why created: to check class balance before model training.

Interpretation: compare bar heights. A visibly smaller class may require class-wise metric attention.

Figure 2. Selected feature distributions

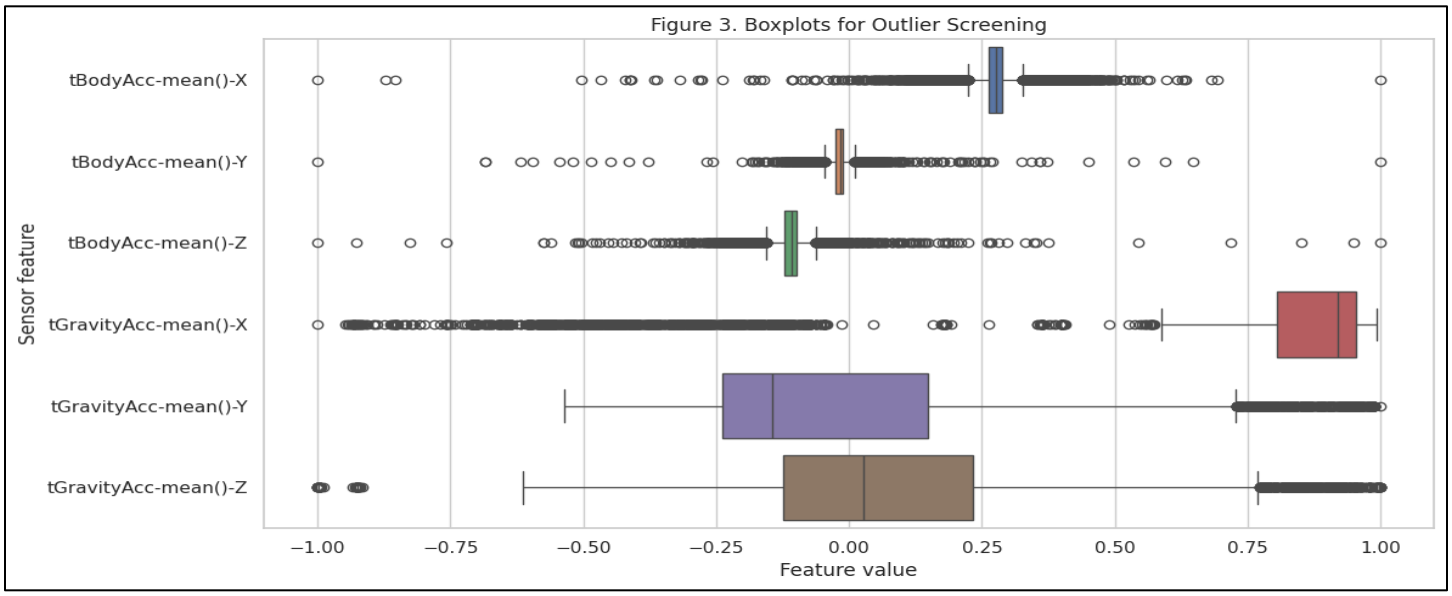


What it shows: frequency distributions of selected sensor-derived features.

Why created: to inspect spread, skewness, concentration, and possible unusual values.

Interpretation: note whether each distribution is symmetric, skewed, concentrated, or multi-peaked.

Figure 3. Boxplots for outlier screening

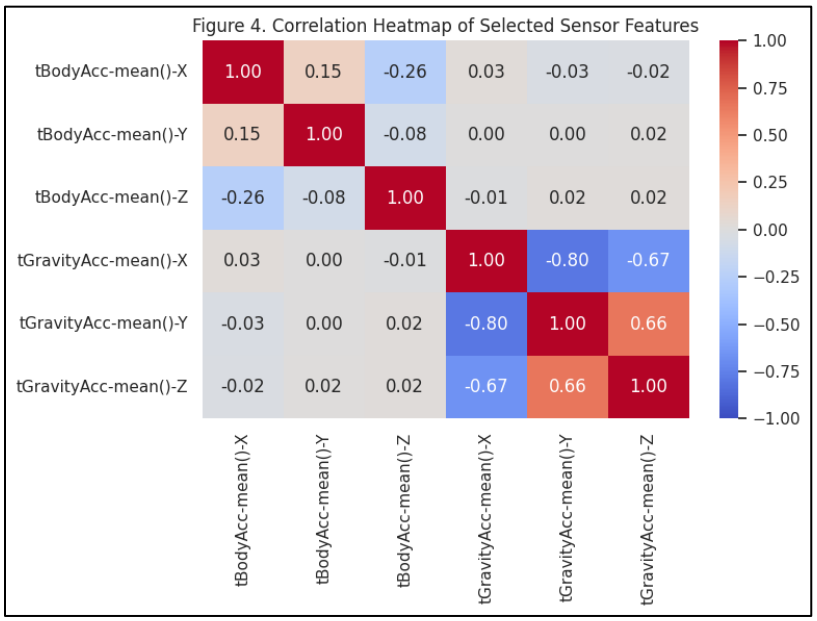


IQR-based outlier counts in selected features:

	outlier_count
tBodyAcc-mean()-X	1795
tBodyAcc-mean()-Y	1122
tBodyAcc-mean()-Z	1180
tGravityAcc-mean()-X	1474
tGravityAcc-mean()-Y	686
tGravityAcc-mean()-Z	607

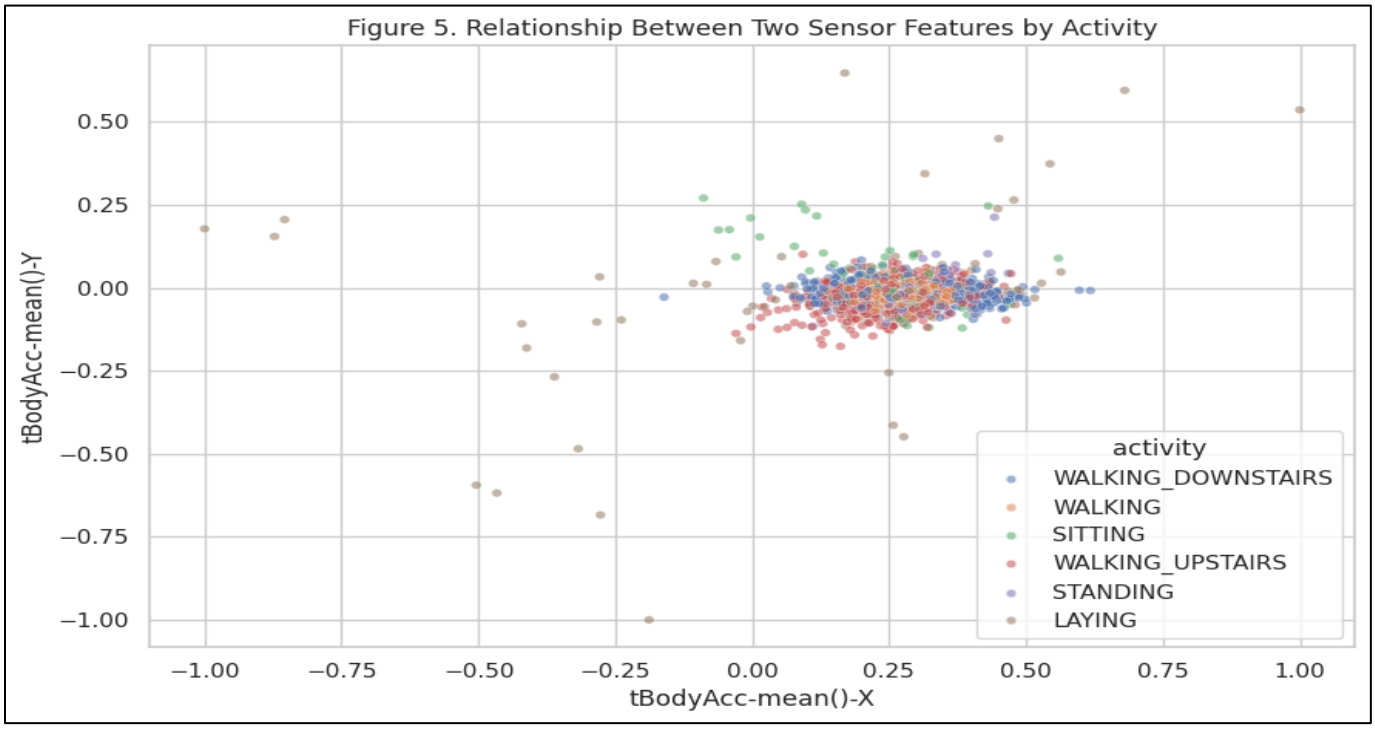
What it shows: median, interquartile range, and points beyond 1.5×IQR whiskers.
Why created: to screen for unusually high or low feature values.
Interpretation: outlier flags are candidates for inspection, not proof of data errors; sensor extremes may be valid.

Figure 4. Correlation heatmap



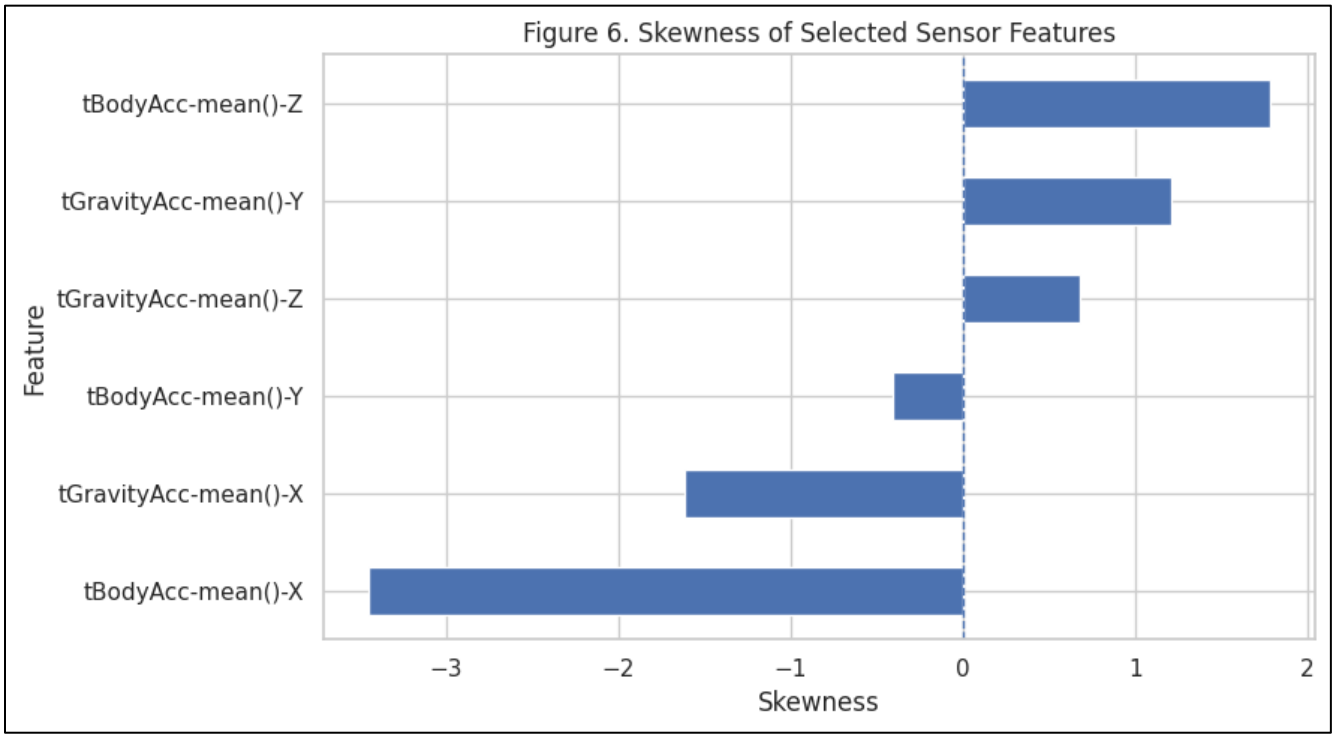
What it shows: pairwise linear correlation between selected features.
Why created: to explore redundancy and relationships among sensor measurements.
Interpretation: values near +1 indicate strong positive linear association; near -1 indicate negative association; near 0 indicate weak linear association.

Figure 5. Feature relationship by activity



What it shows: the relationship between two selected features, with color representing activity.
Why created: to see whether activities occupy different regions in feature space.
Interpretation: clearer separation suggests these features may help classification; overlap suggests more features or nonlinear boundaries are needed.

Figure 6. Feature skewness



What it shows: asymmetry of feature distributions.

Why created: skewness complements histograms with a numerical summary.

Interpretation: positive values indicate right-skew; negative values indicate left-skew; values near zero indicate approximate symmetry.

5. Deep Learning Model

A Multilayer Perceptron (MLP) is used because each record is represented by a fixed-length engineered feature vector. The model includes dense layers with ReLU activation, batch normalization, dropout, and a six-unit softmax output layer. Sparse categorical cross-entropy is used as the loss function, and Adam is used as the optimizer. Two configurations are compared using validation accuracy; early stopping monitors validation loss and restores the best weights.

Selected configuration from runtime: {'units': 256, 'dropout': 0.3, 'learning_rate': 0.0005, 'batch_size': 64}

Best validation accuracy: test set not used for tuning.

6. Model Evaluation and Result Visualizations

Figure 7. Loss versus epoch

Compare training and validation loss. A persistent gap may suggest overfitting; high loss on both sets may suggest underfitting.

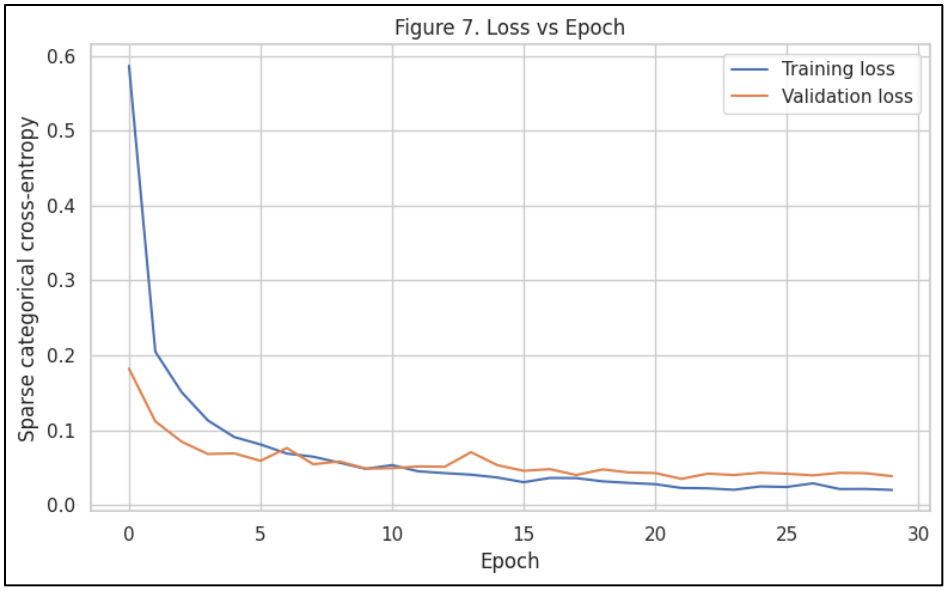
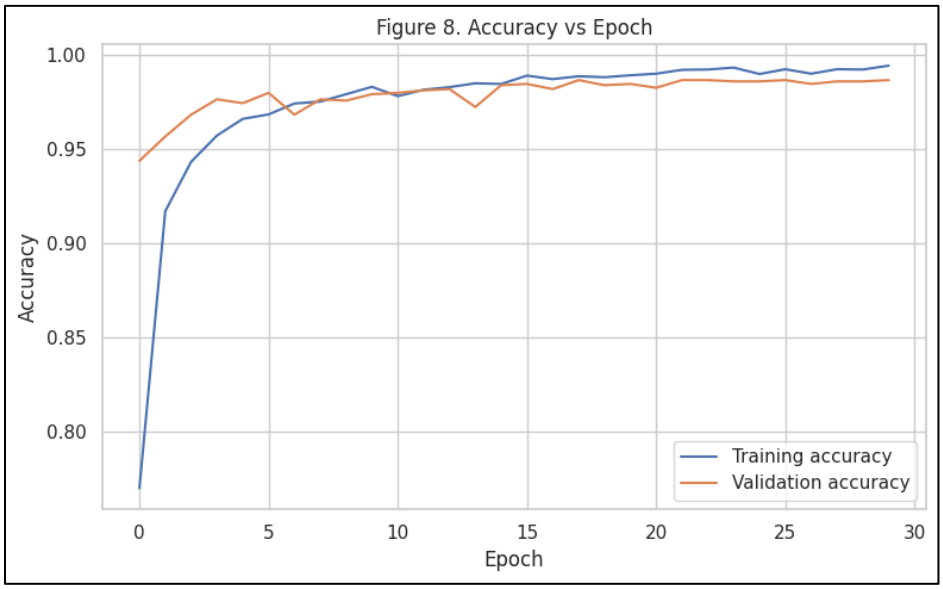


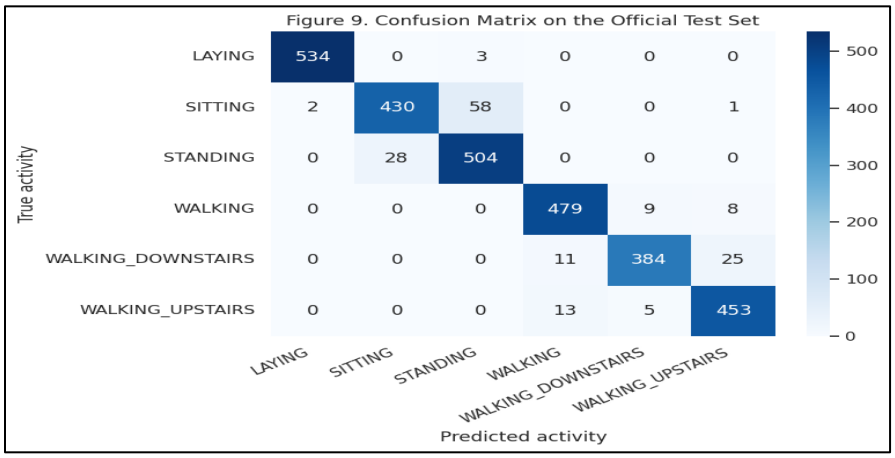
Figure 8. Accuracy versus epoch

Compare training and validation accuracy to assess learning progress and generalization.



TEST ACCURACY: 0.9447

Figure 9. Confusion matrix



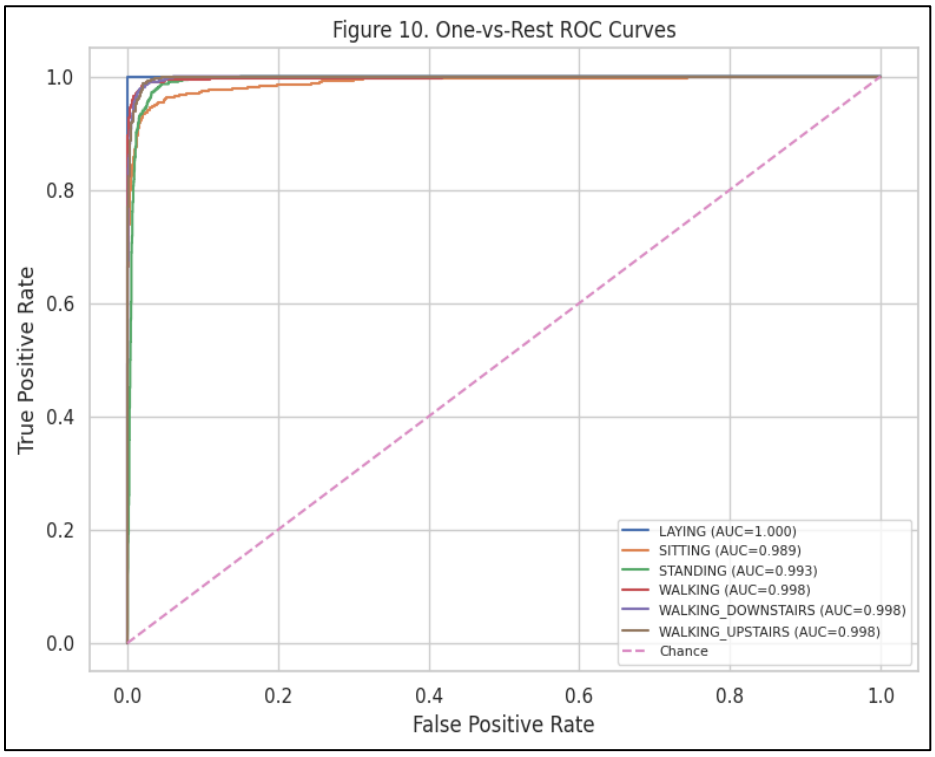
What it shows: correct classifications on the diagonal and errors off the diagonal.

Why created: overall accuracy can hide which activity pairs are confused.

Interpretation: inspect the largest off-diagonal cells and discuss those activity pairs.

Figure 10. Multiclass ROC curves

One-vs-rest ROC curves show the trade-off between true-positive and false-positive rates for each class. AUC summarizes discrimination.

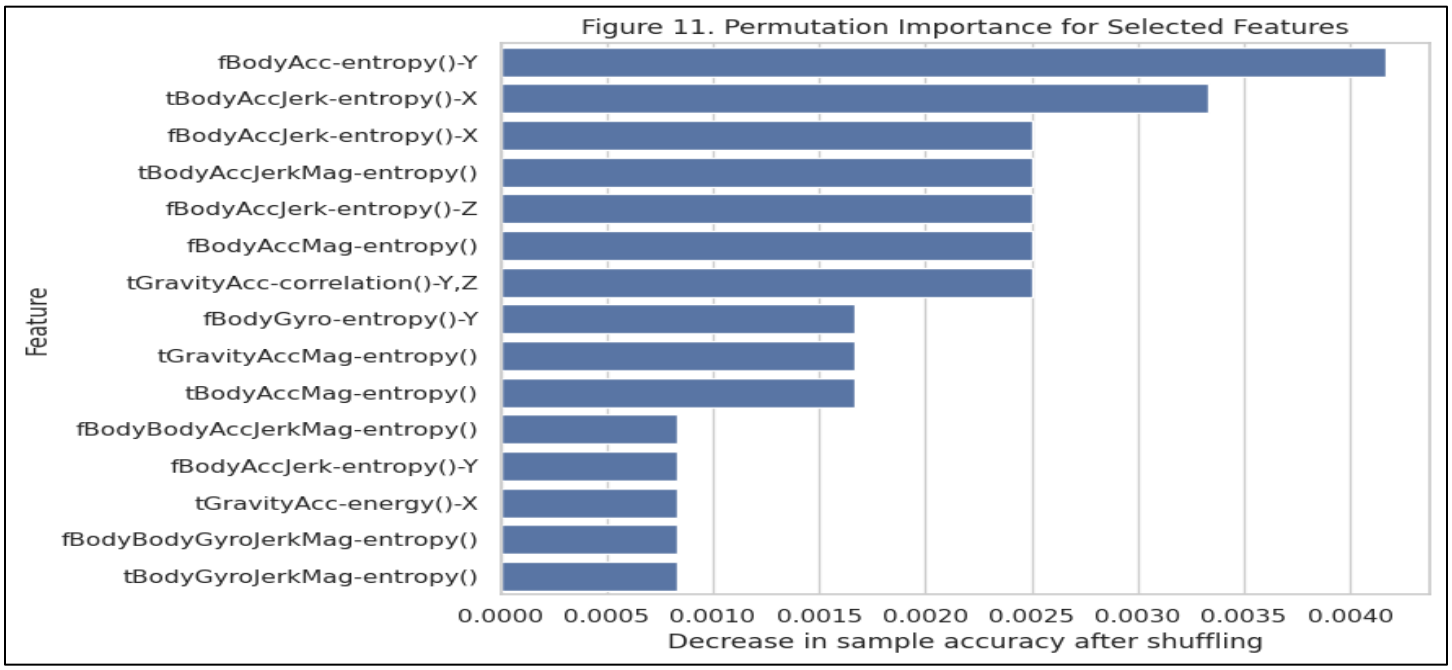


Macro one-vs-rest ROC-AUC: 0.9961

AUC by class: {'LAYING': 0.9999860914717541, 'SITTING': 0.9893647545061929, 'STANDING': 0.9932225750712184, 'WALKING': 0.9978366302101841, 'WALKING_DOWNSTAIRS': 0.9981151186236268, 'WALKING_UPSTAIRS': 0.9982052759570432}

Interpretation: curves nearer the upper-left and larger AUC values indicate

Figure 11. Permutation importance



Interpretation: larger positive accuracy decreases suggest that the model relies more on that feature in this sample. Caution: correlated features can share or mask importance; the result is not causal evidence.

7. Results Summary

- Official test accuracy: 0.9447
- Macro one-vs-rest ROC-AUC: **0.9961**
- Best-performing activity by F1-score: 0.9953
- Activity with lowest F1-score: 0.9062
- Most frequently confused activity pair: Laying - Laying
- Most important selected feature(s): fBodyAcc-entropy()-Y

8. Conclusion and Future Scope

The study demonstrates an end-to-end workflow for exploratory analysis and deep learning classification of smartphone sensor features. The final conclusion should summarize actual distribution patterns, the strongest relationships observed, measured test performance, and any activity classes that are difficult to distinguish. Limitations include the use of engineered rather than raw sensor sequences, possible participant-specific patterns, correlations among features, and the limited scope of the selected feature-importance analysis.

Future scope:

- Use subject-wise validation to test generalization to unseen participants.
- Compare MLP performance with 1D-CNN, LSTM, or hybrid models on raw sensor windows.
- Explore feature selection and dimensionality reduction.
- Evaluate calibration, inference latency, and energy consumption for mobile deployment.
- Test on more diverse participants and real-world sensor recordings.

9. Literature Review

Anguita et al. introduced a public-domain smartphone dataset and discussed activity recognition from embedded sensor signals. Their work provides the dataset basis for this project. Scikit-learn documentation provides established preprocessing and evaluation tools, while TensorFlow/Keras provides the neural-network implementation used for the MLP. The present project focuses on exploratory visualization and a neural-network baseline rather than claiming a new recognition method.

10. References

[1] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A Public Domain Dataset for Human Activity Recognition Using Smartphones," in Proc. ESANN, 2013.

[2] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "Human Activity Recognition on Smartphones Using a Multiclass Hardware-Friendly Support Vector Machine," in Proc. International Workshop on Ambient Assisted Living, 2012.

[3] UCI Machine Learning Repository, "Human Activity Recognition Using Smartphones." https://archive.ics.uci.edu/dataset/240/humanactivityrecognitionusingsmartphones

[4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[5] TensorFlow, "Keras: The Python Deep Learning API." https://keras.io/

[6] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

Appendix A. Code and Reproducibility

The complete commented Python notebook is supplied separately as HAR_EDA_DeepLearning.ipynb. It downloads the dataset, performs preprocessing and EDA, trains and evaluates the model, saves plots, and exports a JSON results summary.

Appendix B. Seven Stages of Visualization

1. Problem framing: define activity recognition and target labels.
2. Data acquisition : retrieve the UCI dataset.
3. Data preparation : inspect, clean, split, and scale the data.
4. Exploratory analysis: summarize distributions and relationships.
5. Visual encoding : select charts suited to categorical and continuous variables.
6. Interpretation and communication: explain each figure in context.
7. Evaluation and refinement: assess model results and identify next steps.